

LAB 4 LẬP TRÌNH CLIENT – SERVER SỬ DỤNG GIAO THỨC TCP

1 MỤC ĐÍCH.....	2 2
YÊU CẦU.....	2 3
HƯỚNG DẪN.....	2
3.1 BƯỚC 1: TẠO DỰ ÁN C# TRONG VISUAL STUDIO	2
3.2 BƯỚC 2: VIẾT MÃ CHO SERVER.....	2
3.3 BƯỚC 3: VIẾT MÃ CHO CLIENT.....	4
3.4 BƯỚC 4: CHẠY THỬ ỨNG DỤNG.....	5
4 HƯỚNG MỞ RỘNG VÀ CẢI TIẾN.....	6

Bài Lab môn Lập Trình Mạng Trang **1/6**

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

1 Mục đích

- Thực hiện kết nối giữa **Client** và **Server** thông qua giao thức TCP/IP, trao đổi dữ liệu giữa chúng, và hiển thị kết quả. Server sẽ lắng nghe yêu cầu kết nối từ client, nhận dữ liệu và trả về kết quả cho client.
- **Ứng dụng thực tế:** Bài thực hành này có thể là nền tảng để xây dựng các ứng dụng mạng, game online, hệ thống chat, hoặc các ứng dụng cần giao tiếp qua mạng.

2 Yêu cầu

1. **Môi trường phát triển:** Cài đặt **Visual Studio** hoặc bất kỳ IDE nào hỗ trợ C#. 2.

Cài đặt .NET Framework: Cài đặt .NET Framework 4.5 hoặc cao hơn. 3. **Giao**

thức TCP: Sử dụng TCP cho kết nối mạng giữa Client và Server. 4. **Giao diện**

console: Sử dụng ứng dụng console để giao tiếp giữa Client và Server.

5. **Địa chỉ IP và Port:** Sử dụng địa chỉ 127.0.0.1 (localhost) để test trên máy tính của bạn, hoặc có thể mở rộng ra các máy khác.

3 Hướng dẫn

3.1 Bước 1: Tạo dự án C# trong Visual Studio

1. Tạo dự án Server:

- Mở Visual Studio.
- Chọn **Create a new project** → **Blank Solution** → đặt tên DemoTCP
- Click phải chuột lên Solutuon DemoTCP → Add → **New Project**
- Chọn **Console App (.NET Core)**.
- Đặt tên cho dự án là **TCPServer** và nhấn **Create**.

2. Tạo dự án Client:

- Tạo dự án mới tương tự, đặt tên là **TCPClient**.

3.2 Bước 2: Viết mã cho Server

1. Mã nguồn Server (TCPServer):

```
using System.Collections.Generic; using System; System;
using System.Collections.Generic;
using System.Linq; System.Linq;
```

Bài Lab môn Lập Trình Mạng Trang 2/6

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

```
using System.Net; using System.Net.Sockets; System.Net.Sockets;
using System.Net;
using System.Text; System.Text;
using System.Threading.Tasks; using System.Threading.Tasks;
```

```
namespace namespace TCPServer
{
    internal class Program
    {
        static void Main( string [] args)
        {
            Console .InputEncoding = Encoding .UTF8;
```

```

        Console.WriteLine(Encoding
        .UTF8;
        // Đợi IP và port mà Server lắng
        nghe
        string ipAddress = "127.0.0.1";
        int port = 12345;

```

```

        // Tạo đối tượng TcpListener để
        lắng nghe kết nối TcpListener
        listener = new TcpListener ( IPAddress.Parse(ipAddress), port);
        listener.Start();

```

```

        Console.WriteLine(
        $"Server đang lắng
        nghe tại {ipAddress} : {port} ...");

```

```

        // Loop vô hạn để chờ
        nhận kết nối
        while ( true )
        {

```

```

            // Chờ
            nhận kết
            nối từ Client
            TcpClient client = listener.AcceptTcpClient();
            Console.WriteLine( "Client đã kết
            nối.");

```

```

        // Lấy luồng giao tiếp
        giữa Server và Client NetworkStream
        stream = client.GetStream();

```

```

        // Đợi đọc dữ liệu
        từ Client
        byte [] buffer = new byte [256];

```

```

        int bytesRead;
        while ((bytesRead = stream.Read(buffer, 0, buffer.Length)) != 0)

```

```

        {
            string dataReceived = Encoding
            .ASCII.GetString(buffer, 0, bytesRead);
            Console.WriteLine(
            $"Nhập dữ liệu từ client:
            {dataReceived}");

```

```

            // Gửi lại dữ liệu
            cho Client

```

```

        }
        string response =
            "Server đã nhận: " + dataReceived;
        byte [] dataToSend = Encoding.ASCII.GetBytes(response);
        stream.Write(dataToSend, 0,
            dataToSend.Length); }

        // Đóng kết nối
        client.Close();
        Console.WriteLine(
            "Đóng kết nối");
    }
}
}

```

Bài Lab môn Lập Trình Mạng Trang 3/6

Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

3.3 Bước 3: Viết mã cho Client

▪ Mã nguồn Client (TCPClient):

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Net.Sockets;
using System.Text;
using System.Threading.Tasks;

namespace TCPClient
{
    internal class Program
    {
        static void Main( string [] args)
        {
            Console.InputEncoding = Encoding.UTF8;
            Console.OutputEncoding = Encoding.UTF8;

            string
            serverAddress = "127.0.0.1"; // Địa chỉ IP của Server
            int port = 12345; // Cổng mà Server đang lắng nghe

            // Tạo đối tượng TcpClient để kết nối
        }
    }
}

```

Server

TcpClient client = new TcpClient (serverAddress, port);

Console.WriteLine(\$"Đã kết nối tới server {serverAddress} : {port} ");

// Lắng nghe giao tiếp

Server

NetworkStream stream = client.GetStream();

while (true)
{

// Nhập thông điệp từ bàn phím

Console.WriteLine("Nhập thông điệp gửi tới server: ");

string message = Console.ReadLine();

// Nếu người dùng nhập "exit", thoát khỏi chương trình if (message.ToLower() == "exit")

{
Console.WriteLine("Đóng kết nối với server.");
break ;
}

// Gửi dữ liệu lên Server
byte [] dataToSend = Encoding .ASCII.GetBytes(message);

stream.Write(dataToSend, 0, dataToSend.Length);
Console.WriteLine(\$"Đã gửi: {message} ");

// Nhập phản hồi từ Server
byte

[] buffer = new byte [256];
int bytesRead = stream.Read(buffer, 0, buffer.Length);
string response = Encoding .ASCII.GetString(buffer, 0, bytesRead);
Console.WriteLine(\$"Nhập từ server: {response} "); }

// Đóng kết nối
stream.Close();
client.Close();

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Đã đóng k ế t n ố");
        }
    }
}
```

3.4 Bước 4: Chạy thử ứng dụng

Sử dụng tính năng "Multiple Startup Projects" trong Visual Studio

Visual Studio cho phép bạn cấu hình để nhiều dự án (project) trong một solution có thể chạy đồng thời khi nhấn nút "Start" (F5). Đây là cách dễ dàng để chạy cả TCPServer và TCPClient đồng thời.

Bước 1: Mở Solution Explorer

- Mở Visual Studio và tải dự án của bạn.
- Mở Solution Explorer (Nếu chưa thấy, vào menu View -> Solution Explorer).

Bước 2: Cấu hình chạy đồng thời

1. Chọn cả hai dự án trong Solution:

- Nhấp chuột phải vào Solution trong Solution Explorer (nằm ở góc trên bên trái, bên ngoài của các dự án).
- Chọn Properties.

2. Cấu hình "Multiple Startup Projects":

- Trong cửa sổ Solution Properties, chọn tab Startup Project.
- Chọn Multiple startup projects.
- Đối với TCPServer, chọn Start trong cột "Action".
- Đối với TCPClient, chọn Start trong cột "Action".

3. Lưu cấu hình:

- Nhấn OK để lưu và đóng cửa sổ cấu hình.

Bước 3: Chạy Solution

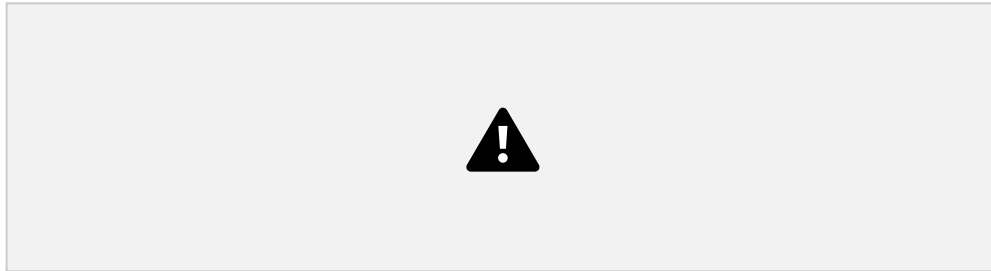
- Khi bạn nhấn Start (F5), cả hai dự án sẽ được chạy đồng thời:

Bài Lab môn Lập Trình Mạng Trang 5/6

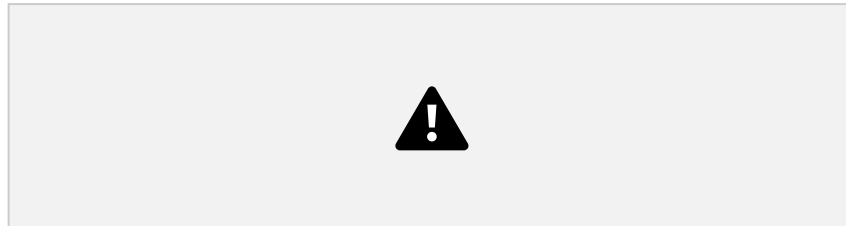
Trường Đại học Công Thương TP. Hồ Chí Minh Khoa Công nghệ Thông tin

- TCPServer sẽ khởi động và bắt đầu lắng nghe các kết nối từ TCPClient.
- TCPClient sẽ khởi động và kết nối tới server mà bạn vừa thiết lập.

Kết quả từ Server:



Kết quả từ Client:



4 Hướng mở rộng và cải tiến

Hiện tại, ứng dụng chưa có cơ chế xử lý lỗi và ngoại lệ. Trong môi trường thực tế, các vấn đề như mất kết nối mạng, server không phản hồi hoặc client gửi dữ liệu không hợp lệ có thể xảy ra. Cần cải tiến để đảm bảo ứng dụng không bị crash và có thể xử lý tình huống lỗi một cách tốt nhất.

